Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2133 – Estructuras de Datos y Algoritmos

2026 - 1

Taller 2

Objetivos

- Implementar algoritmos de ordenamiento en distintos escenarios.
- Analizar la complejidad temporal segun el tamaño de los datos.
- Modificar algoritmos de sorting para resolver problemas especificos.

Introducción

Después de varias semanas ayudando al Profesor Oak a organizar información sobre los Pokémon, ha surgido un nuevo desafío en el laboratorio. Investigadores de distintas regiones han comenzado a enviar grandes cantidades de datos sobre Pokémon, incluyendo atributos como nivel, velocidad y ataque, entre muchos otros.

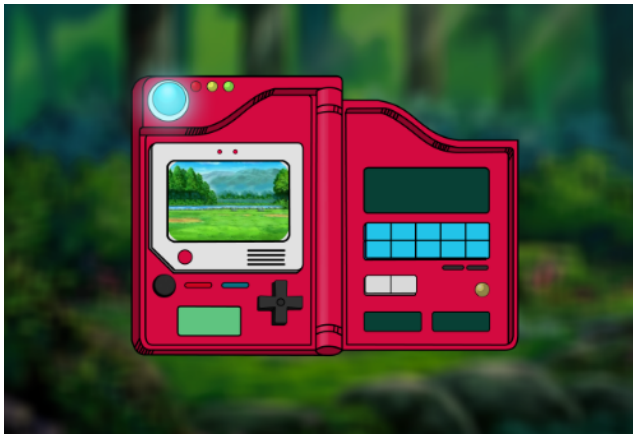
El problema es que toda esta información llega desordenada, y organizarla manualmente podría tomar demasiado tiempo. Para poder analizar correctamente los datos y continuar con las investigaciones, es necesario implementar métodos eficientes que permitan ordenar grandes volúmenes de información de manera rápida.

En este taller, su tarea como ayudante del laboratorio será implementar distintos algoritmos de ordenamiento que permitan organizar los datos de los Pokémon según distintos criterios. A través de estos ejercicios, se busca analizar el comportamiento de distintos algoritmos de sorting y comprender cuándo es conveniente utilizar cada uno de ellos dependiendo del tamaño y características de los datos.

¡Orden en la Pokedex!

En el laboratorio del Profesor Oak se reciben constantemente nuevos registros de Pokemon provenientes de distintas regiones. Cada investigador envía información sobre los Pokemon que ha observado, incluyendo atributos como su ID en la pokedex, su nivel y su velocidad.

Debido a la gran cantidad de datos recibidos, la información llega completamente desordenada. Para poder analizar correctamente estos registros y compararlos entre distintas investigaciones, el laboratorio necesita ordenar los datos de forma eficiente.



Problema

El Profesor Oak ha reunido varios registros de Pokemon en arreglos de datos. Dependiendo del tamaño del conjunto de datos, distintos algoritmos de ordenamiento pueden ser más o menos eficientes.

Utilizando tus conocimientos de programación, deberás implementar distintos algoritmos de sorting para ordenar los registros del laboratorio y facilitar su análisis.

Entidades

Pokemon: Cada Pokémon cuenta con un `ID` único, un valor de `nivel` y un valor de `velocidad`. Todos los atributos son valores numéricos.

Flujo

Primero recibirás una línea con el entero E, que representa la cantidad de eventos a procesar.

Luego se recibirán E eventos, los cuales corresponden a distintas operaciones que el laboratorio del Profesor Oak necesita realizar sobre los registros de Pokemon.

Cada evento tendrá el nombre que indica el tipo de operación a ejecutar. Dependiendo del tipo de evento, se entregará la información necesaria para realizar la operación correspondiente.

Los eventos deberán ser procesados en el mismo orden en que aparecen en el input.

Para los eventos que requieran ordenar información, recibirás un entero N que representa la cantidad de Pokémon a procesar, seguido de N líneas con la información correspondientes al Pokemon.

Dependiendo del tipo de evento, deberás aplicar el algoritmo de ordenamiento correspondiente y mostrar el resultado solicitado.

Parte Formativa - Eventos

`SMALL_SORT {N} {atr} {orden}`

En este evento deberás ordenar los N Pokémon que se te entregaran según el atributo indicado en `atr`, que puede ser `NIVEL` o `VELOCIDAD`. Junto a el parámetro `orden`, que puede ser `ASC` o `DESC`, indicando si el ordenamiento debe realizarse de forma ascendente o descendente. Todo esto siguiendo con N lineas de los Pokemon a registrar con sus respectivos `ID`, `NIVEL`, `VELOCIDAD`.

En caso de empate en ambos atributos seleccionados, deberás desempatar con respecto al orden entregado utilizando el `ID` del Pokémon, el cual es único.

Este evento corresponde a conjuntos de datos pequeños, por lo que se espera que utilices un algoritmo de ordenamiento con complejidad aproximada $O(n^2)$.

El resultado esperado debe mostrar el siguiente output:

output

```
{N} Pokemones ordenados por {atr} en forma {orden}:
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
    .
    .
    .
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
```

Donde atr podria ser tanto VELOCIDAD como NIVEL segun el caso.

`BIG_SORT {N} {atr_1} {atr_2} {orden}`

En este evento deberás ordenar los N Pokémon que se te entregaran según el primer atributo indicado en `atributo_1`, que puede ser `NIVEL` o `VELOCIDAD`. En caso de empate con el primer atributo se debera hacer la comparacion con el segun atributo `atributo_2`, que tambien puede ser `NIVEL` o `VELOCIDAD`. Todo junto a el parámetro `orden`, que puede ser `ASC` o `DESC`, indicando si el ordenamiento debe realizarse de forma ascendente o descendente. FInalmente siguiendo con N lineas de los Pokemon a registrar con sus respectivos `ID`, `NIVEL`, `VELOCIDAD`.

En caso de empate en ambos atributos seleccionados, deberás desempatar con respecto al orden entregado utilizando el `ID` del Pokémon, el cual es único.

Este evento corresponde a conjuntos de datos grandes, por lo que se espera que utilices un algoritmo de ordenamiento con complejidad aproximada $O(n * logn)$ para que sea eficiente.

El resultado esperado debe mostrar el siguiente output:

output

```
{N} Pokemones ordenados por {atr\_1} {atr\_2} en orden {orden}:
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    .
    .
    .
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
```

Se puede suponer que `atr_1` y `atr_2` siempre seran distintos.

Ejecución

Tu programa se debe compilar con el comando `make` y debe generar un ejecutable de nombre `pokemones` que se ejecuta con el siguiente comando:

```
./pokemones input output
```

donde `input` será un archivo con los eventos a simular y `output` el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind deberás utilizar el siguiente comando:

```
valgrind ./pokemones input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

El archivo de input comenzará con el número `E` que corresponde al número de eventos a recibir, y las `E` líneas correspondientes a cada uno. Para cualquiera de los dos eventos, se entregara tambien un numero `N`, que indicara la cantidad de pokemones a registrar a continuacion.

Como ejemplo tenemos el siguiente input:

input

```
3 # Cantidad de eventos
SMALL_SORT 5 NIVEL ASC # el 5 seria la cantidad de pokemones a registrar
1 10 23 # primer pokemon a registrar de los 5
5 12 67
4 1 22
6 56 20
10 5 5 # ultimo pokemon a registrar de los 5
BIG_SORT 1000 VELOCIDAD NIVEL DESC
20 21 22 # primer pokemon a registrar de los 1000
11 105 22
.
.
568 31 11 # ultimo pokemon a registrar de los 1000
SMALL_SORT 2 VELOCIDAD DESC
7 7 7
8 9 7
```

Output

```
output
5 Pokemones ordenados por NIVEL en orden ASC:
    Pokemon ID 4: NIVEL 1
    Pokemon ID 10: NIVEL 5
    Pokemon ID 1: NIVEL 10
    Pokemon ID 5: NIVEL 12
    Pokemon ID 6: NIVEL 56
1000 Pokemones ordenados por VELOCIDAD NIVEL en orden DESC:
    Pokemon ID 11: VELOCIDAD 22 NIVEL 105 # Como hay un empate
    Pokemon ID 20: VELOCIDAD 22 NIVEL 21  # gana el de mayor nivel
    .
    .
    .
    Pokemon ID 568: VELOCIDAD 11 NIVEL 31
2 Pokemones ordenados por VELOCIDAD en orden DESC:
    Pokemon ID 8: VELOCIDAD 7 # Mayor ID gana por empate
    Pokemon ID 7: VELOCIDAD 7
```

Evaluación (Solo parte evaluada)

La nota de tu taller es calculada a partir de testcases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán testcase publicos para que puedas probar tu implementación durante el taller.

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 2 segundos** y utilizar menos de 1 GB de RAM[1], de lo contrario recibirás 0 puntos en ese test. Ademas, se evaluará de que no existan leaks/errores de memoria. Por último, está **estrictamente prohibido** el uso de la función `realloc`, cuyo uso implicará un descuento de 5 décimas.

Integridad académica

Este curso se adscribe al Código de Honor establecido por la Escuela de Ingeniería. Todo trabajo evaluado en este curso debe ser hecho **individualmente** por el alumno y **sin apoyo de terceros**. Se espera que los alumnos mantengan altos estándares de honestidad académica, acorde al Código de Honor de la Universidad. Cualquier acto deshonesto o fraude académico está prohibido; los alumnos que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada alumno conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Pregrado de la Escuela de Ingeniería.

Uso de IA

Se prohíbe el uso de herramientas de inteligencia artificial para la realización de esta tarea. Esto incluye, pero no se limita al uso de modelos de lenguaje como ChatGPT, Copilot, u otras tecnologías similares para la generación automática de código, soluciones, o cualquier parte del trabajo requerido. Nos reservamos el derecho de revisar todas las tareas entregadas con el fin de detectar el uso de inteligencia artificial en la creación de las soluciones. Las tareas en las que se determine que se ha utilizado IA serán consideradas como una infracción a la política de honestidad académica y serán tratadas como casos de copia.

[1] Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`